

Aplicación web a desarrollar: MapaHamburguesa : plataforma colaborativa para descubrir, mapear y reseñar hamburgueserías en la Provincia de Buenos Aires. (nombre por definir)

Grupo: Lucas Rodrigo Cardozo, Matias Larregle y Facundo Tarizzo

ADR-001 · Supabase como capa de datos, autenticación y almacenamiento

- Estado: Aceptado.
- Contexto: El prototipo requiere una base de datos PostgreSQL, autenticación mediante Google OAuth y almacenamiento de fotografías de los locales. Debido a las restricciones de presupuesto y al carácter académico del proyecto, se buscó una solución que permitiera concentrar estos servicios en una misma plataforma y minimizar el tiempo de configuración. Se evaluó una arquitectura compuesta por DigitalOcean Managed PostgreSQL, DigitalOcean Spaces y Auth0, pero fue descartada debido a la necesidad de integrar tres servicios independientes y configurar manualmente la autenticación. También se consideró Firebase, aunque se descartó por su mayor dependencia del ecosistema de Google y por ofrecer una menor afinidad con el stack basado en Python y FastAPI.
- Decisión: Se selecciona Supabase en su plan gratuito como Backend as a Service. La plataforma proporciona una base de datos PostgreSQL estándar, autenticación mediante Google OAuth y almacenamiento de archivos dentro de una misma solución, reduciendo la complejidad de implementación y facilitando futuras migraciones.
- Consecuencias:
 - Ventajas: Configuración rápida de autenticación, almacenamiento compatible con S3 y una interfaz visual para administrar la base de datos.
 - Tradeoffs: Dependencia de un único proveedor para tres componentes críticos del sistema. Una eventual indisponibilidad de Supabase afectaría simultáneamente la autenticación, los datos y las imágenes almacenadas. Sin embargo, los límites del plan gratuito son suficientes para las necesidades del MVP.

ADR-002 · Despliegue de la API y Entorno de Cómputo VPS

- **Estado:** Aceptado.

- **Contexto:** El backend de la aplicación requiere un entorno permanente y accesible mediante una URL pública para garantizar su disponibilidad durante el período de evaluación. Debido a que la lógica de negocio se implementa en FastAPI y necesita mantener conexiones persistentes con Supabase, se analizaron distintas alternativas de despliegue. Se descartó Render en su plan gratuito porque suspende las aplicaciones después de períodos de inactividad, lo que podría afectar una demostración académica. También se evaluó Vercel, pero se descartó debido a que no ofrece soporte nativo para una aplicación FastAPI ejecutándose como servidor ASGI persistente. Finalmente, las funciones serverless de DigitalOcean fueron consideradas inadecuadas porque la aplicación requiere mantener estado durante el ciclo de cada request y una conexión estable con la base de datos.
- **Decisión:** Se opta por desplegar la aplicación utilizando FastAPI en un Droplet de DigitalOcean, aprovechando los créditos disponibles del programa Student Pack. La instancia ejecuta Ubuntu y utiliza Nginx como proxy inverso junto con Gunicorn y Uvicorn para servir la aplicación.
- **Consecuencias:**
 - **Ventajas:** Disponibilidad 24/7, control total sobre el entorno de ejecución y una infraestructura sencilla de justificar como arquitectura cloud dentro del proyecto académico.
 - **Tradeoffs:** La configuración inicial requiere tareas adicionales relacionadas con Nginx y los certificados SSL. Estas tareas se mitigan mediante el uso de Certbot y Let's Encrypt. En caso de agotarse los créditos estudiantiles de DigitalOcean, la aplicación puede migrarse a Railway con cambios mínimos en la configuración.

ADR-003 · Hosting del frontend y distribución de archivos estáticos

- **Estado:** Aceptado.
- **Contexto:** El prototipo posee una interfaz desarrollada con HTML y JavaScript (Leaflet.js) y necesita distribuir tanto el frontend como las imágenes asociadas a los locales mediante URLs públicas con tiempos de respuesta bajos. Se evaluó utilizar Vercel (se hizo la ui previamente para barajar la estética de la misma) para alojar la interfaz, pero se descartó porque incorporaba un proveedor adicional sin aportar beneficios relevantes desde el punto de vista académico. También se consideró servir los archivos estáticos directamente desde el Droplet, aunque esta alternativa fue desestimada debido a que agregaría carga innecesaria al servidor de aplicación.

- **Decisión:** Se selecciona DigitalOcean Spaces aprovechando su compatibilidad con almacenamiento de objetos y CDN integrada. El servicio se utiliza tanto para alojar el frontend estático como para almacenar las fotografías de los locales, consolidando ambos requerimientos dentro del ecosistema de DigitalOcean.
- **Consecuencias:**
 - **Ventajas:** Un único servicio cubre el almacenamiento y la distribución del frontend y de las imágenes, incorpora CDN sin configuración adicional y mantiene compatibilidad con el estándar S3.
 - **Tradeoffs:** Implica un costo adicional respecto al Droplet, aunque este se encuentra cubierto por los créditos estudiantiles durante la duración del proyecto. Para controlar el consumo de almacenamiento, las imágenes son redimensionadas antes de ser cargadas al sistema.

ADR-004 · Orquestación del asistente de onboarding y elección del LLM

- **Estado:** Aceptado.
- **Contexto:** El sistema incorpora un asistente conversacional que ayuda a los usuarios a descubrir locales según sus preferencias. Para ello se requiere un Modelo de Lenguaje Grande capaz de generar respuestas en lenguaje natural a partir de los resultados obtenidos desde la base de datos. Dado que el proyecto posee un presupuesto nulo, se evaluaron alternativas que pudieran utilizarse sin necesidad de asociar tarjetas de crédito. Se descartó OpenAI debido a la fricción asociada a los requisitos de faenor madurez de su ecosicturación y también se consideró Groq, aunque sus límites de uso y la mstema de desarrollo lo hicieron menos conveniente para el prototipo.
- **Decisión:** Se selecciona la API de Gemini mediante Google AI Studio. El backend traduce las preferencias del usuario en filtros sobre la información almacenada en Supabase y posteriormente utiliza Gemini únicamente para sintetizar y redactar una respuesta en lenguaje natural a partir de los resultados recuperados.
- **Consecuencias:**
 - **Ventajas:** Acceso gratuito sin necesidad de tarjeta de crédito, buena ventana de contexto y facilidad de integración mediante el SDK oficial para Python.
 - **Tradeoffs:** Dependencia de una API externa y latencia condicionada por el tráfico del proveedor. En caso de agotarse la cuota gratuita, el asistente puede desactivarse sin afectar las funcionalidades principales de la aplicación.

ADR-005 · Servicio de mailing transaccional

- **Estado:** Aceptado.
- **Contexto:** El flujo colaborativo de la aplicación genera eventos que justifican el envío de notificaciones al usuario, como la aprobación de un local sugerido o la generación de resúmenes periódicos con actividad reciente. Se necesitaba una solución de correo electrónico que pudiera integrarse fácilmente con el backend y mantenerse dentro de un presupuesto cero. Se evaluó SendGrid, pero fue descartado debido a la mayor complejidad del proceso de configuración y verificación de dominio. También se descartó la posibilidad de enviar correos directamente desde el Droplet, ya que la falta de reputación de dominio incrementa considerablemente la probabilidad de que los mensajes sean considerados spam.
- **Decisión:** Se utiliza Resend en su plan gratuito, integrándolo mediante una llamada REST desde FastAPI para gestionar las notificaciones generadas por la aplicación.
- **Consecuencias:**
 - **Ventajas:** Integración sencilla, funcionamiento dentro del free tier y ausencia de requisitos de tarjeta de crédito.
 - **Tradeoffs:** Dependencia de un servicio externo y necesidad de verificar un dominio remitente antes del despliegue. Las notificaciones son opcionales y la aplicación conserva todas sus funcionalidades incluso cuando el usuario decide no utilizarlas.

ADR-006 · Protección contra tráfico automatizado

- Estado: Aceptado.
- Contexto: La aplicación se encuentra expuesta mediante una URL pública durante el período de evaluación. Tanto el asistente basado en Gemini como las funcionalidades colaborativas podrían verse afectadas por tráfico automatizado o intentos de abuso capaces de consumir los recursos gratuitos disponibles. Se evaluó implementar mecanismos de limitación directamente dentro de FastAPI, pero esta alternativa fue descartada debido a que el tráfico seguiría llegando al servidor antes de ser filtrado, sin ofrecer una protección efectiva sobre la cuota consumida por los servicios externos.
- Decisión: Se incorpora Cloudflare en su plan gratuito como proxy inverso delante del Droplet. Adicionalmente, se utiliza Turnstile para proteger los endpoints más sensibles relacionados con la generación de contenido y las reseñas.
- Consecuencias:

- Ventajas: El servidor no expone directamente su dirección IP y el sistema incorpora una capa adicional de protección frente a tráfico automatizado sin introducir fricción significativa para los usuarios reales.
- Tradeoffs: Se añade un servicio externo adicional que requiere configuración de DNS y agrega una pequeña latencia al recorrido de las peticiones. Al tratarse de una decisión complementaria de seguridad, no se considera uno de los servicios cloud principales contemplados por la rúbrica.

